



К(П)ФУ

ИНСТИТУТ МАТЕМАТИКИ И МЕХАНИКИ

Кафедра аэрогидромеханики

Казанцев Р.В.

Казань

2024

Содержание

1	Установка программ	2
1.1	Установка Python	2
1.2	Установка Git	3
1.3	Установка Visual Studio Code	3
2	Выполнение домашнего задания №0	3
2.1	Обновление репозитория перед выполнением домашнего задания	5
2.1.1	Первоначальная настройка	5
2.1.2	Обновление перед выполнением нового домашнего задания	6
3	Задание №1. Размытие изображения	6
3.1	Часть 1. Добавление padding	6
3.1.1	Задание	7

1 Установка программ

Перед выполнением домашнего задания необходимо установить Python, Git и редактор кода.

1.1 Установка Python

Скачайте Python с официального сайта:

<https://www.python.org/downloads/windows/>

1. Выберите **Download Windows installer (64-bit)**.
2. В окне установки обязательно поставьте галочку:

☐ **Add python.exe to PATH.**

3. После установки откройте терминал PowerShell и выполните команду:

```
python --version
```

Должна быть выведена установленная версия Python, например:

```
Python 3.13.15
```

Если команда не работает или отображается неправильная версия, попробуйте перезагрузить компьютер.

4. Проверьте установленную версию pip:

```
python -m pip --version
```

Например, результат может выглядеть следующим образом:

```
pip 26.2.1 from
↪ C:\Users\docto\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip
↪ (python 3.13)
```

5. Обновите pip:

```
python -m pip install --upgrade pip
```

6. Проверьте, откуда запускается Python:

```
where.exe python
```

1.2 Установка Git

Скачайте Git с официального сайта:

<https://git-scm.com/install/windows>

После установки откройте PowerShell и выполните:

```
git --version
```

Например, результат может выглядеть следующим образом:

```
git version 2.55.0.windows.5
```

1.3 Установка Visual Studio Code

В качестве среды разработки рекомендуется установить Visual Studio Code:

<https://code.visualstudio.com/docs/setup/windows>

После установки откройте раздел **Extensions** и установите расширения:

- Python;
- Git.

После установки всех необходимых программ рекомендуется перезагрузить компьютер.

2 Выполнение домашнего задания №0

1. Делайте форк репозитория курса:

https://github.com/kkazantsevr/geom_course_2026

Процесс создания форка был показан на практическом занятии.

2. Клонировать свой репозиторий на компьютер.

- (a) Создайте папку для хранения учебных проектов. Например, **works**.
- (b) Откройте терминал в созданной папке.

В Windows 11 это можно сделать, нажав правой кнопкой мыши внутри папки и выбрав соответствующий пункт меню для открытия терминала.

- (c) Выполните команду:

```
git clone https://github.com/ВАШ_НИКНЕЙМ_GITHUB/geom_course_2026
```

После выполнения команды будет создана папка курса:

```
/works/geom_course_2026
```

- (d) Перейдите в папку репозитория:

```
cd geom_course_2026
```

(e) Создайте виртуальное окружение Python. В терминале выполните:

```
python -m venv .venv
```

Эта команда создаст папку `.venv`. По умолчанию папки, названия которых начинаются с точки, могут не отображаться в некоторых файловых менеджерах.

Рекомендуется самостоятельно разобраться, зачем необходимо виртуальное окружение Python и какие преимущества оно даёт при разработке программ.

(f) Активируйте виртуальное окружение:

```
..venv\Scripts\Activate.ps1
```

После этого для текущего проекта будет использоваться виртуальное окружение `.venv`. Все дополнительные библиотеки Python будут устанавливаться именно в это окружение.

(g) Установите необходимые библиотеки:

```
python -m pip install numpy pandas matplotlib
```

3. Создайте файл:

```
geom_course_2026/home_work_0/main.py
```

Программа должна выводить:

- версию Python;
- версию NumPy;
- версию Pandas;
- версию Matplotlib.

Запустить программу можно командой:

```
python.exe main.py
```

Это возможно благодаря тому, что ранее было создано и активировано виртуальное окружение Python.

Без виртуального окружения пришлось бы указывать полный путь к интерпретатору Python, например:

```
C:\Users\Ваш_Профиль\AppData\Local\Programs\Python\Python313\python.exe main.py
```

4. Далее необходимо сообщить Git, какие изменения произошли в файловой структуре проекта — какие файлы были добавлены или изменены. Для этого выполните:

```
git add .
```

5. Добавьте комментарий к выбранным изменениям и зафиксируйте их:

```
git commit -m "ура, 1 задание почти завершено"
```

6. Отправьте изменения на GitHub:

```
git push
```

При первом использовании команды может появиться окно с просьбой авторизоваться через браузер и подтвердить адрес электронной почты. В более сложном случае, git в терминале может попросить username и password от github, однако, начиная с 2024 года вместо пароля приходится вводить токен, сгенерированный в настройках аккаунта github.

После успешной авторизации дальнейшая отправка изменений на GitHub будет выполняться автоматически при использовании команды `git push`.

2.1 Обновление репозитория перед выполнением домашнего задания

Исходный репозиторий курса

```
https://github.com/kkazantsevru/geom\_course\_2026
```

будет периодически обновляться. Поэтому перед началом выполнения нового домашнего задания (то есть с домашнего задания номер 1) необходимо обновить свой форк.

2.1.1 Первоначальная настройка

Эти действия необходимо выполнить только один раз. Перейдите в папку репозитория:

```
cd geom_course_2026
```

Посмотрите, какие удалённые репозитории уже подключены:

```
git remote -v
```

После создания форка и клонирования репозитория вы увидите примерно следующее:

```
origin https://github.com/ВАШ_НИКНЕЙМ_GITHUB/geom_course_2026.git (fetch)
origin https://github.com/ВАШ_НИКНЕЙМ_GITHUB/geom_course_2026.git (push)
```

Добавьте исходный репозиторий курса под именем `upstream`:

```
git remote add upstream https://github.com/kkazantsevru/geom_course_2026.git
```

Теперь проверьте список удалённых репозитория:

```
git remote -v
```

Должны появиться два удалённых репозитория:

- `origin` — личный форк студента;
- `upstream` — исходный репозиторий курса.

2.1.2 Обновление перед выполнением нового домашнего задания

Перед началом каждого нового домашнего задания выполните:

```
git fetch upstream
```

Эта команда загружает информацию о новых изменениях из исходного репозитория курса. Обновите локальную копию:

```
git merge upstream/main
```

После этого отправьте обновления в свой форк на GitHub:

```
git push origin main
```

Таким образом, последовательность команд перед началом нового домашнего задания будет следующей:

```
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

После выполнения этих команд локальная копия репозитория и личный форк на GitHub будут содержать последние изменения из исходного репозитория курса.

3 Задание №1. Размытие изображения

В этом задании необходимо реализовать простейший фильтр размытия изображения с помощью библиотеки NumPy.

Изображение будем рассматривать как двумерный в NumPy. Чёрно-белое изображение имеет размер

(N, M) . Каждый элемент изображения содержит значение яркости соответствующего пикселя.

Например, для черно-белого

```
image[i, j] = 255
```

означает, что пиксель с индексами (i, j) имеет белый цвет.

3.1 Часть 1. Добавление padding

При обработке изображения часто необходимо обращаться к пикселям, находящимся около границы изображения.

Рассмотрим изображение

```
1 2 3
4 5 6
7 8 9
```

Если добавить `padding` шириной 2, получим:

```
0 0 | 0 0 0 | 0 0
0 0 | 0 0 0 | 0 0
----|-----|----
0 0 | 1 2 3 | 0 0
0 0 | 4 5 6 | 0 0
0 0 | 7 8 9 | 0 0
----|-----|----
0 0 | 0 0 0 | 0 0
0 0 | 0 0 0 | 0 0
```

То есть вокруг исходного изображения добавляется рамка заданной ширины, заполненная нулями. Возможен и другой вариант, например, `padding`-элементы заполняются крайними строками и столбцами:

```
1 1 | 1 2 3 | 3 3
1 1 | 1 2 3 | 3 3
----|-----|----
1 1 | 1 2 3 | 3 3
4 4 | 4 5 6 | 6 6
7 7 | 7 8 9 | 9 9
----|-----|----
7 7 | 7 8 9 | 9 9
7 7 | 7 8 9 | 9 9
```

3.1.1 Задание

1. Допишите функцию `pad_image` в файле

```
task1/blur.py
```

Сигнатура функции:

```
def pad_image(image: np.ndarray, pad: int, mode: str) -> np.ndarray:
    ...
```

Функция принимает 3 аргумента:

- `image` — двумерный массив NumPy с изображением;
- `pad` — ширина добавляемой рамки,
- `mode` — способ реализации `padding`.

Функция должна вернуть новое изображение, окружённое рамкой из нулей шириной `pad` если `mode = "zero"`, либо изображение окруженное повторяющимися строками и столбцами как в примере, если `mode = "edge"`. Исходное изображение должно находиться в центре результирующего массива.

2. Допишите функцию

```
def box_blur(image: np.ndarray, kernel_size: int, mode: str) -> np.ndarray:  
    ...
```

Эта функция должна реализовать "метод скользящего окна" для размытия изображения. Очевидно, `kernel_size` должен быть нечетным числом, а `pad = kernel_size // 2`.

3. Запустите функцию `main()` командой `python.exe main.py`.